

AI - Powered Trading Analysis System

A PROJECT REPORT

Submitted by

Vaishnani Kush

220090107203

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

Computer Engineering

C. K. Pithawala College of Engineering and Technology, Surat



Gujarat Technological University, Ahmadabad

May, 2026



C. K. Pithawala College of Engineering and Technology

Opposite Surat Airport, Behind DPS School, Near Malvan Mandir, Dumas Road, Surat

CERTIFICATE

*This is to certify that the project report submitted along with the project entitled **Modular Algo Trading Analysis System** has been carried out by **Vaishnani Kush** under my guidance in partial fulfillment for the degree of Bachelor of Engineering in Computer Engineering, 8th Semester of Gujarat Technological University, Ahmedabad during the academic year 2025-26.*

Prof. Stephanie Tamakuwala
Internal Guide

Dr. Vishruti Desai
Head of the Department

TO WHOM IT MAY CONCERN

Date: 01/05/2026

This is to certify that **Mr. Vaishnani Kush Nileshbhai** has successfully completed his internship in the field of **Python With Data Science (AI/ML)** from **02-02-2026** to **01-05-2026** under the guidance of Mr. Darshan Chauhan.

His internship activities included brainstorming and designing Machine Learning Algorithms and AI solutions using various python frameworks and libraries.

During his internship program with us, he was exposed to various processes and demonstrated diligence, hard work, and inquisitiveness.

We wish him every success in his life and career.

For **Durvasa Infotech**

Darshan Chauhan
Director



+917573818618

www.durvasainfotech.com



C. K. Pithawala College of Engineering and Technology

Opposite Surat Airport, Behind DPS School, Near Malvan Mandir, Dumas Road, Surat

DECLARATION

I hereby declare that the Internship / Project report submitted along with the Internship / Project entitled **Modular Algo Trading Analysis System** submitted in partial fulfillment for the degree of Bachelor of Engineering in Computer Engineering to Gujarat Technological University, Ahmedabad, is a bonafide record of original project work carried out by me at **Durvsha Infotech** under the supervision of **Prof. Stephanie Tamakuwala** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

Vaishnani Kush
220090107203

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of my internship project titled “Modular Algo Trading Analysis System using Python, Streamlit, and MetaTrader 5.”

First and foremost, I am deeply thankful to my internship organization for providing me with the opportunity to work on a real-world, industry-oriented project. This experience has significantly enhanced my practical knowledge in algorithmic trading, financial data analysis, and application development using Python and Streamlit.

I would like to extend my heartfelt appreciation to my project guide and mentors for their continuous support, valuable suggestions, and constructive feedback at every stage of the project. Their guidance helped me understand real-world trading systems, broker integration, and best practices for building modular and scalable applications.

I am also grateful to my faculty members and institution for encouraging me to take up this project and providing the necessary resources and environment to complete it successfully.

This project has been a great learning experience, allowing me to strengthen my technical skills and gain hands-on experience in developing a multi-timeframe trading system, integrating MetaTrader 5 for automated trading, and designing an interactive dashboard for real-time market analysis and trade execution.

Vaishnani Kush

Abstract

The Modular Algo Trading Analysis System is an AI-powered Python-based application developed for real-time market analysis and automated trading using MetaTrader 5 (MT5). The system is designed to help traders make intelligent trading decisions through machine learning models, technical indicators, and automated execution.

The project uses a modular architecture where different components such as market data collection, indicator calculation, AI signal generation, risk management, trade execution, and dashboard interface are separated into independent modules for scalability and maintainability.

The system integrates technical indicators such as Exponential Moving Average (EMA), Relative Strength Index (RSI), and Moving Average Convergence Divergence (MACD). In addition, a Random Forest Machine Learning model is used to improve Buy/Sell decision accuracy.

A professional user interface is developed using Streamlit with real-time charts, multi-timeframe signals, trade history, performance monitoring, and risk-reward management. The system supports assets like Bitcoin and Gold and can execute trades automatically through MetaTrader 5 demo account.

Overall, this project demonstrates the practical implementation of AI in algorithmic trading by combining finance, automation, and machine learning techniques.

List of Figures

Figure 1 History of Company	1
Figure 2 Organization Chart	3
Figure 3 Sequence of operation	7
Figure 4 Gantt chart	14
Figure 5 Process Flow	17
Figure 6 State Transition	24
Figure 7 Dashboard	28

List of Tables

Table 1 Scope of Work	2
Table 2 Plant Capacity	4
Table 3 Technical Specification	6
Table 4 Effort and Time	12
Table 5 Cost Estimation	13
Table 6 Technology Specification	27
Table 7 Comparison of System	29
Table 8 Technical approach	31
Table 9 Test cases	33
Table10 Problems and solutions	35

Table of Contents

Acknowledgement	i
Abstract	ii
List of Figures	iii
List of Tables	iv
Table of Contents	v
1 Overview of the Company	1
1.1 History	1
1.2 Different product / scope of work	2
1.3 Organization chart.	3
1.4 Capacity of plant	3
2 Company Overview	5
2.1 Department working	5
2.2 Technical specifications	6
2.3 Sequence of operations	6
2.4 Production Stages	7
3 Introduction to Internship	9
3.1 Project / Internship Summary	9
3.2 Purpose	10
3.3 Objective	10
3.4 Scope	10
3.5 Technology and Literature Review	11
3.6 Project / Internship Planning	11
3.6.1 Approach with Justification	11
3.6.2 Cost Estimation	12
3.6.3 Roles	13
3.6.4 Group Dependencies	13
3.7 Project / Internship Scheduling	14
4 System Analysis	15
4.1 Study of Current System	15
4.2 Problem and Weaknesses of Current System	15
4.3 Requirements of New System	15

4.4 System Feasibility	15
4.4.1 Contribution to the objective.	15
4.4.2 Constraints	15
4.4.3 Integration	16
4.5 Process Flow	16
4.6 Features	17
4.7 Modules	18
4.8 Methodology	19
5 System Design	20
5.1 System Design & Methodology	20
5.2 Design Details	21
5.3 Interface Design	23
5.3.1 State Transition	24
5.3.2 Forms and Reports	25
5.3.3 Security Considerations	25
6 Implementations	26
6.1 Environment	26
6.2 Technology Specifications	26
6.3 Results	27
6.4 Result Analysis	28
7 Testing	30
7.1 Plan	30
7.2 Test Results	32
7.2.1 Test Cases	33
8 Conclusion and Discussion	34
8.1 Overall Analysis	34
8.2 Visit by Mentor	34
8.3 Evaluation Dates	34
8.4 Problem and Solutions	35
8.5 Summary	35
8.6 Future Scope	35
References	37

1 OVERVIEW OF THE COMPANY

1.1 History

Durvasa Infotech is a growing IT services and digital solutions company headquartered in Surat, Gujarat, India. The company was established in 2019 with the vision of providing innovative and result-driven digital solutions to businesses.

The organization was founded by industry professionals with the aim of bridging the gap between traditional business operations and modern digital transformation needs. Initially focused on web development services, the company gradually expanded into multiple domains such as mobile application development, digital marketing, SEO, and UI/UX design.

Over the years, the company has grown into a multi-service IT solutions provider, serving small to medium-sized enterprises and helping them build a strong online presence. It emphasizes quality service, client satisfaction, and continuous technological innovation

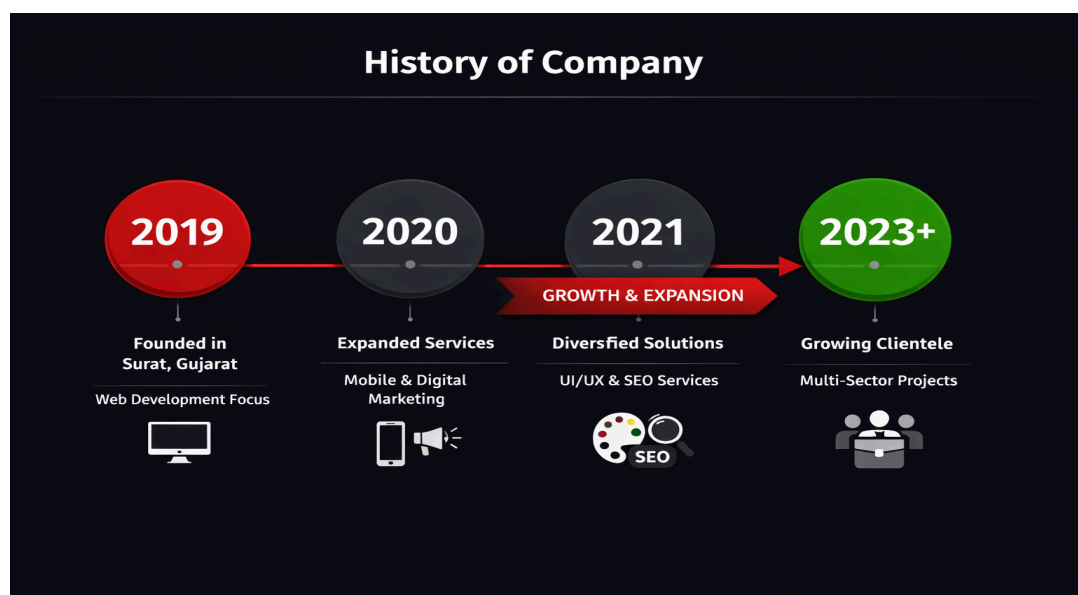


Figure 1 History of Company

1.2 Different product / scope of work

Durvasa Infotech offers a wide range of IT and digital services that cover the entire lifecycle of software and business growth solutions. The company focuses on delivering customized, scalable, and user-friendly solutions to its clients.

Service	Description	Purpose
Web Development	Development of responsive and dynamic websites using modern technologies	Build strong online presence
Mobile App Development	Android application development with user-friendly UI	Enhance mobile accessibility
Digital Marketing	SEO, SEM, Google Ads, Social Media Marketing	Increase brand visibility
UI/UX Design	Designing intuitive and attractive interfaces	Increase brand visibility
Web Hosting	Secure and reliable hosting services	Ensure performance & uptime
Content Writing	Professional content for websites and branding	Improve engagement
Training & Internship	Practical IT training programs	Skill development for students

Table 1 Scope of Work

1.3 Organization chart.

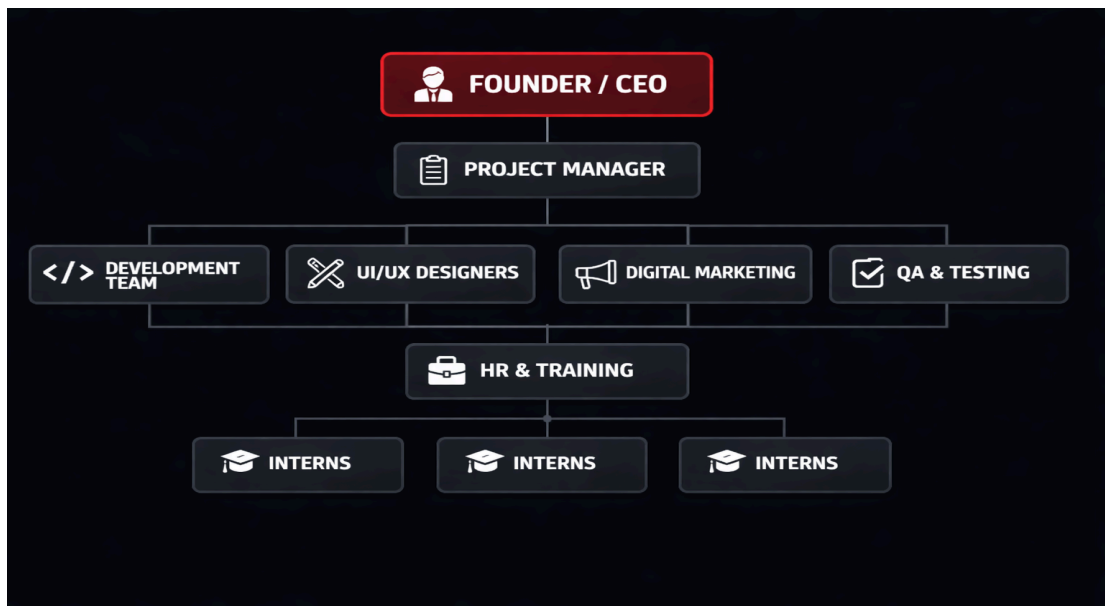


Figure 2 Organization Chart

1.4 Capacity of plant

Although Durvasa Infotech is not a manufacturing company, its “plant capacity” can be understood in terms of IT infrastructure, workforce, and project handling capabilities.

Header 1	Header 2	Header 3
Office Location	Surat, Gujarat	Main operational center
Team Size	10–50 employees	Agile and scalable workforce
Technology Stack	Web, Mobile, Digital Marketing Tools	Modern development environment
Project Capacity	Multiple concurrent projects	Efficient project management
Training Facilities	Internship & training programs	Skill development support

Development Tools	React, Node.js, Android, SEO tools	Industry-standard tools
Client Support	Ongoing maintenance & support	Long-term client relationship
Network & Systems	High-speed internet & cloud tools	Smooth operations

Table 2 Plant Capacity

2 COMPANY OVERVIEW

Durvasa Infotech is an IT service-based organization that focuses on delivering software solutions rather than manufacturing physical products. The company operates through various departments that collaboratively work on designing, developing, testing, and deploying software applications.

The workflow of the company follows a structured Software Development Life Cycle (SDLC), ensuring high-quality product delivery. The major departments include Development, Design, Testing, Marketing, and HR/Training.

2.1 Department working

Each department in the organization plays a crucial role in the successful delivery of software projects.

1. Development Department

- Responsible for building web and mobile applications
- Works on frontend (React.js) and backend (Node.js, Express.js)
- Integrates APIs and handles database operations (MySQL)
- Implements business logic and system functionality

2. UI/UX Design Department

- Designs user interfaces and user experience
- Creates wireframes, prototypes, and layouts
- Ensures responsive and visually appealing design
- Maintains design consistency across the application

3. QA & Testing Department

- Performs manual and automated testing
- Identifies bugs and ensures bug-free delivery
- Conducts API testing using tools like Postman
- Ensures software meets quality standards

4. Digital Marketing Department

- Handles SEO, social media marketing, and online campaigns
- Promotes applications and services
- Improves user engagement and traffic

5. HR & Training Department

- Manages recruitment and employee training
- Provides internship programs and skill development
- Monitors employee performance and growth

2.2 Technical specifications

The company uses modern tools and technologies for efficient software development.

Department	Tools / Technologies Used	Purpose
Development	React.js, Node.js, Express.js, MySQL	Web application development
Backend APIs	REST APIs, JSON	Data communication
Database	MySQL	Data storage and management
UI/UX Design	Figma, Adobe XD	Interface design
Version Control	Git, GitHub	Code management
Training & Internship	Python, AI-ML, Data Analytics	AI & ML Model
Marketing	Google Ads, SEO Tools	Promotion & analytics

Table 3 Technical specification

2.3 Sequence of operations

The company follows a structured process for developing software applications. The sequence of operations is as follows:

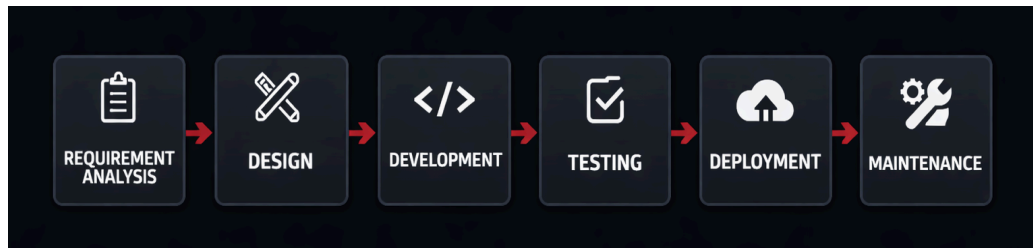


Figure 3 Sequence of operation

2.4 Production Stages

In IT companies, “production” refers to **software development stages**. Each stage is critical for delivering a high-quality product.

1. Requirement Analysis

- Understanding client needs and project goals
- Gathering functional and non-functional requirements
- Creating documentation

2. Design Stage

- Creating UI/UX designs using tools like Figma
- Designing system architecture
- Planning database schema

3. Development Stage

- Frontend development using React.js
- Backend development using Node.js and Express.js
- API creation and integration
- Database connectivity with MySQL

4. Testing Stage

- Performing unit testing and integration testing
- API testing using Postman
- Fixing bugs and improving performance

5. Deployment Stage

- Hosting the application on servers
- Configuring databases and backend services
- Ensuring application accessibility

6. Maintenance Stage

- Monitoring system performance
- Fixing issues and bugs
- Adding new features and updates

3 INTRODUCTION TO INTERNSHIP

The internship at Durvasa Infotech provided practical exposure to real-world applications of Artificial Intelligence and Machine Learning along with modern software development practices. During this internship, I worked on developing two projects: “Modular Algo Trading Analysis System” and “AI Code Generator.”

The first project focused on building a system for real-time market analysis and automated trading using Python, Streamlit, and MetaTrader 5. The second project involved creating an AI-based application that generates code based on user input, improving productivity and automation in programming tasks.

Both projects were designed to be modular, scalable, and user-friendly, helping me understand system design, API integration, and real-time data processing.

This experience helped me gain practical knowledge in AI/ML concepts, Python development, and building real-world applications while improving my problem-solving and development skills.

3.1 Project / Internship Summary

The Modular Algo Trading Analysis System is an AI-powered trading application developed using Python, Streamlit, and MetaTrader 5. The system performs real-time market analysis, generates intelligent trading signals, and supports automated trade execution in a demo trading environment.

During the internship, I was responsible for:

- Designing modular architecture for trading system
- Developing interactive dashboard using Streamlit
- Implementing technical indicators (EMA, RSI, MACD)
- Integrating Machine Learning model for AI signals
- Connecting MetaTrader 5 API for automated trading
- Implementing risk management using Stop Loss / Take Profit
- Creating real-time charts and performance analytics
- Testing, debugging, and improving system performance.

3.2 Purpose

The main purpose of this internship project is to:

- Develop an intelligent algorithmic trading system using Python
- Apply Artificial Intelligence and Machine Learning in finance
- Automate market analysis and trade execution
- Bridge theoretical knowledge with practical implementation
- Provide real-time trading assistance using technical indicators

3.3 Objective

The key objectives of the project are:

- To design a modular and scalable trading platform
- To build an interactive dashboard using Streamlit
- To implement indicators such as EMA, RSI, and MACD
- To integrate Random Forest ML model for predictions
- To connect MetaTrader 5 API for auto trading
- To perform multi-timeframe market analysis
- To manage risk using Stop Loss and Take Profit
- To ensure reliable performance and error handling

3.4 Scope

What the System Can Do:

- Analyze real-time market data for Bitcoin and Gold
- Generate Buy / Sell / Wait signals using AI logic
- Perform multi-timeframe market analysis
- Calculate EMA, RSI, and MACD indicators
- Use Machine Learning model for improved decision making
- Execute trades automatically using MetaTrader 5 demo account
- Manage Stop Loss and Take Profit levels

What the System Cannot Do:

- Limited to demo trading environment
- Not guaranteed for profit in live markets
- Supports selected assets only
- No mobile application version currently
- Advanced deep learning models not implemented yet

3.5 Technology and Literature Review

The project is developed using modern technologies related to Python programming and financial market analysis.

- Python is used as the core programming language for implementing trading logic and system modules.
- Streamlit is used to build an interactive and user-friendly dashboard for displaying trading signals and market data.
- MetaTrader 5 (MT5) API is used for connecting the system with the trading platform and executing trades.
- Pandas is used for data handling and analysis.
- Technical Indicators (EMA, RSI, MACD) are used for market analysis and signal generation.

The system is based on algorithmic trading concepts where trading decisions are made using predefined rules and indicators instead of manual intervention.

3.6 Project / Internship Planning

The project was planned and developed in a structured manner by dividing it into different modules such as data analysis, signal generation, trade execution, and user interface.

3.6.1 Approach with Justification

Project / Internship Development Approach and Justification

The project follows a modular development approach, where each component of the system is developed separately.

- Modular approach makes the system easy to maintain and extend

- Each module (analysis, execution, UI) works independently
- Easy debugging and testing of individual components
- Suitable for real-time trading systems

This approach ensures better scalability and flexibility for future improvements.

3.6.2 Cost Estimation

Since this is an internship project, the cost is estimated based on development effort, tools, and resources.

Effort & Time Estimation:

Task	Estimated Time
Requirement Analysis	2 days
System Design (Modular)	3 days
Dashboard Development (UI)	5 days
Trading Logic Implementation	7 days
MT5 API Integration	6 days
Testing & Debugging	5 days
Final Improvements	2 days
Total Duration	30 days (approx.)

Table 4 Effort & Time Estimation

Cost Estimation:

Component	Estimated Cost (INR)
Development Cost (Intern Level)	₹12,000
Software Tools (Python, MT5, etc.)	₹0

Internet & System Usage	₹2,000
Total Estimated Cost	₹14,000

Table 5 Cost Estimation

3.6.3 Roles

Roles and Responsibilities

- Designed overall system architecture
- Developed trading logic and indicators
- Built Streamlit dashboard for user interaction
- Integrated MetaTrader 5 API for trading
- Performed testing and debugging
- Handled errors related to broker and execution

3.6.4 Group Dependencies

This project was developed individually, so there were no group dependencies.

However, the project depends on:

- MetaTrader 5 platform for trading execution
- Internet connection for real-time data
- External libraries such as Streamlit and Pandas

3.7 Project / Internship Scheduling

Gantt Chart:

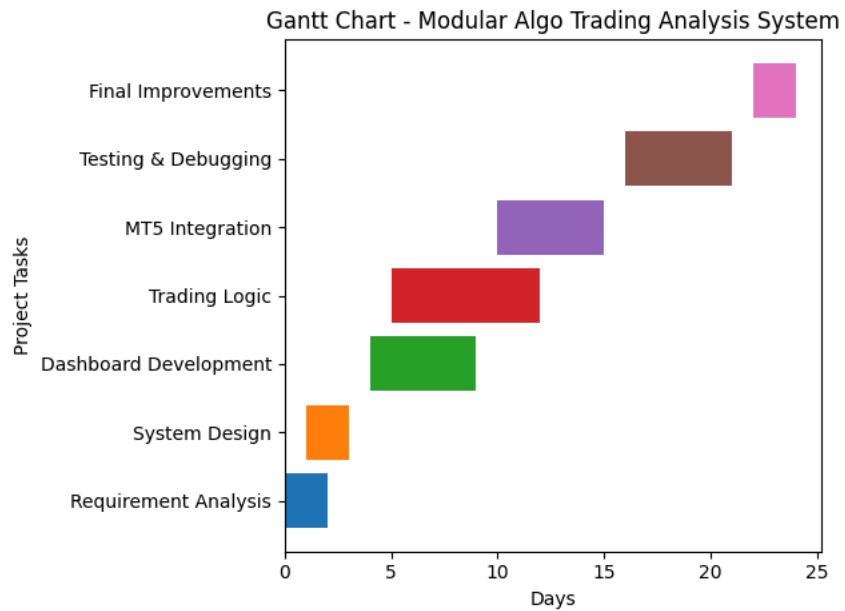


Figure 4 Gantt Chart

4 SYSTEM ANALYSIS

4.1 Study of Current System

The current trading system is mostly manual, where traders analyze charts and make decisions based on experience and basic indicators. There is limited automation, and most decisions depend on human judgment.

4.2 Problem and Weaknesses of Current System

- Time-consuming manual analysis
- High chances of human error
- Lack of real-time automated decision making
- No proper integration with trading platforms for auto execution
- Difficult to monitor multiple timeframes and indicators simultaneously

4.3 Requirements of New System

- Automated market analysis using technical indicators
- Real-time data processing
- Integration with MetaTrader 5 for trade execution
- Interactive dashboard for monitoring signals
- Modular and scalable system design
- Error handling and reliable performance

4.4 System Feasibility

4.4.1 Contribution to the objective.

The system helps in achieving the objective by automating trading analysis and execution, reducing manual effort and improving decision-making.

4.4.2 Constraints

- Depends on internet connection and MT5 platform
- Limited to demo trading environment
- Requires basic system configuration for running Python and Streamlit

4.4.3 Integration

The system can be integrated with MetaTrader 5 and can be extended to other APIs or trading platforms in the future.

4.5 Process Flow

The proposed system (**Modular Algo Trading Analysis System**) follows a structured process flow that ensures smooth interaction between the user, system modules, and the trading platform (MetaTrader 5).

User Side Process Flow:

- User Opens Dashboard (Streamlit Application)
- Connect to MetaTrader 5 Account
- Select Trading Symbol (e.g., XAUUSD, EURUSD)
- View Real-Time Market Data
- Analyze Technical Indicators (EMA, RSI, MACD)
- Monitor Buy/Sell Signals
- Choose Trade Action (Manual / Auto)
- Execute Trade (Open Position)
- Monitor Open Trades
- Close Trade / Exit Position

System (Backend) Process Flow:

- Fetch Real-Time Market Data
- Calculate Technical Indicators
- Generate Trading Signals (Buy/Sell)
- Validate Trading Conditions
- Send Trade Request to MetaTrader 5
- Execute Order (Buy/Sell)
- Monitor Trade Status
- Handle Errors (Connection / Execution Issues)
- Update Dashboard with Trade Details

Admin / Control Flow (Optional System Control):

- Start / Stop Trading System
- Monitor System Performance
- Check Logs and Errors
- Manage Trade Execution Settings (SL/TP)
- Debug and Maintain System

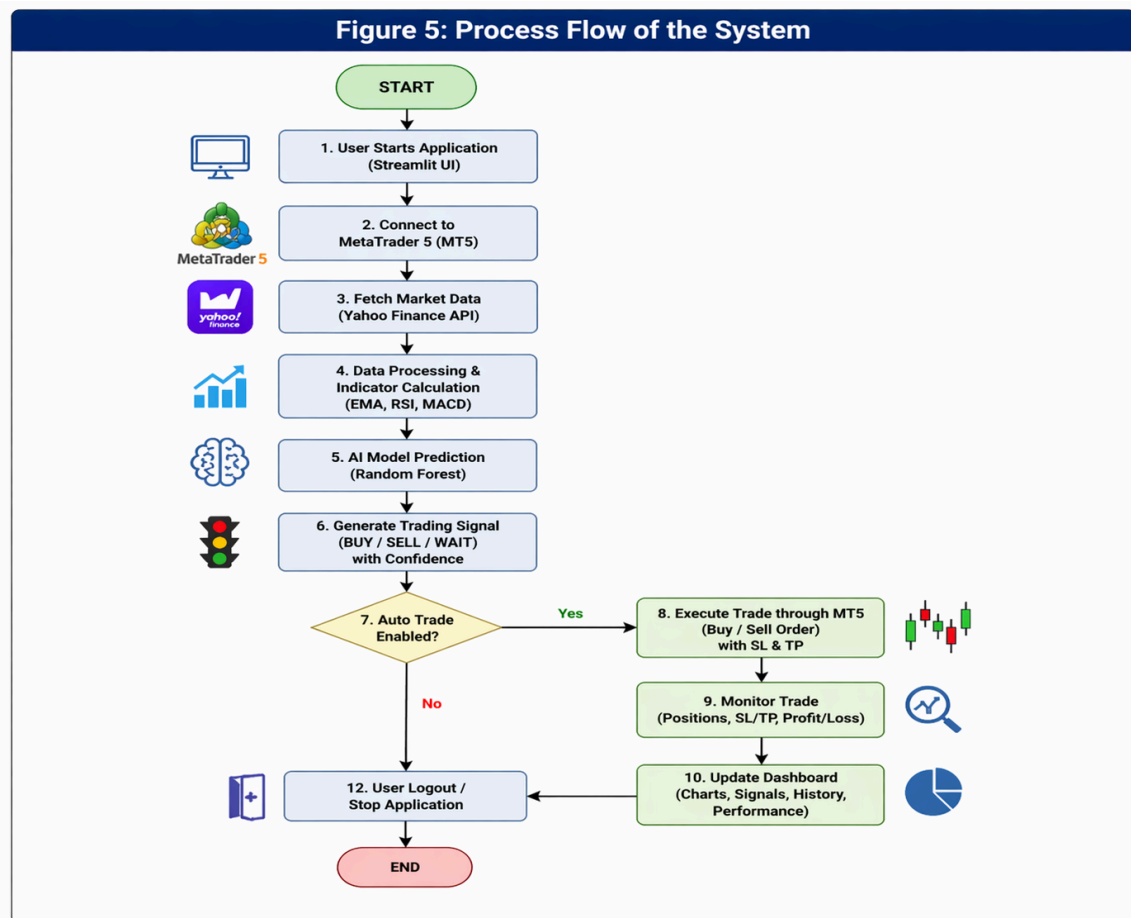


Figure 5 Process Flow

4.6 Features

The proposed system includes modern AI-based features to improve trading efficiency and user experience.

User Features:

- Professional dashboard using Streamlit

- Real-time market price monitoring
- Bitcoin and Gold asset selection
- Multi-timeframe signal analysis
- Technical indicators (EMA, RSI, MACD)
- Buy / Sell / Wait intelligent signals
- Manual trade execution
- Automatic trade execution
- Trade history tracking
- Performance monitoring

AI Features:

- Random Forest Machine Learning model
- AI Decision Engine
- Confidence-based signal logic
- Trend analysis using indicators

System Features:

- MetaTrader 5 integration
- Live Yahoo Finance data support
- Risk Management (SL / TP)
- Fast signal processing
- Modular architecture
- Easy scalability

4.7 Modules

The system is divided into the following main modules:

- **Market Data Module**
Fetches real-time price data from APIs and MT5.
- **Indicator Module**
Calculates EMA, RSI, and MACD.

- **AI Prediction Module**
Uses Random Forest model for Buy/Sell decisions.
- **Signal Generation Module**
Combines indicators and AI output.
- **Trade Execution Module**
Executes trades through MetaTrader 5.
- **Risk Management Module**
Controls Stop Loss and Take Profit.
- **Dashboard Module**
Displays charts, signals, history, and metrics.
- **Error Handling Module**
Handles API, connection, and execution errors.

4.8 Methodology

The project follows a **modular and real-time system development approach**.

- **Hardware:**
Personal computer with internet connection
- **Software:**
Python, Streamlit, MetaTrader 5, Pandas
- **Algorithms / Techniques:**
Indicator-based trading using EMA, RSI, and MACD
- **Methodology:**
Modular design approach for easy development and maintenance
- **Approach:**
Real-time data processing and automated trade execution

Justification:

This methodology is chosen because it is simple, efficient, and suitable for real-time trading systems. The modular approach makes the system scalable, easy to debug, and adaptable for future improvements such as AI-based trading.

5 SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, and data flow of the system. It ensures that the application is efficient, scalable, and meets both functional and non-functional requirements.

5.1 System Design & Methodology

The Modular Algo Trading Analysis System is designed using a modular layered architecture where each component performs a specific task independently. This improves maintainability, scalability, and performance.

Architecture Used: Modular Layered Architecture

1. Presentation Layer (User Interface)

- Developed using Streamlit
- Provides interactive dashboard
- Displays charts, indicators, signals, and trade controls

2. Application Layer (Trading Logic)

- Developed in Python
- Calculates EMA, RSI, MACD
- Generates Buy / Sell / Wait signals

3. AI Layer

- Uses Random Forest Machine Learning model
- Enhances decision making using trend + RSI logic
- Confidence-based recommendations

4. Integration Layer

- Connects with MetaTrader 5 API
- Executes trades automatically
- Manages order opening and closing

5. Data Layer

- Uses Yahoo Finance live market data
- Uses Pandas DataFrames for processing

Methodology Used:

- Modular Development Approach
- Incremental Testing
- Real-time Data Processing
- Continuous Improvement

Design Principles:

- Modularity
- Scalability
- Efficiency
- Reliability
- User-Friendly Interface

5.2 Design Details

1. Data Structure Design

The system uses real-time market data instead of a traditional database. Data is processed and handled dynamically using structured formats.

- Market Data (Price, Time, Symbol)
- Indicator Data (EMA, RSI, MACD values)
- Trade Data (Order type, price, SL, TP)

Structure Used:

- Pandas DataFrames for data analysis
- Dictionary / JSON format for API communication
- Real-time tick data from MetaTrader 5

2. Process Design

The system processes data in a step-by-step flow:

User Action → Data Fetching → Indicator Calculation → Signal Generation → Trade Execution → Dashboard Update

Example: Trading Process

- User selects symbol
- System fetches real-time data
- Indicators (EMA, RSI, MACD) are calculated
- Buy/Sell signal is generated
- Trade request sent to MT5
- Trade executed and displayed on dashboard

3. Structure Design

The project follows a modular file structure:

project/

├── app.py (Streamlit UI)

├── mt5_connect.py

├── mt5_trade.py

├── mt5_helper.py

├── mt5_auto.py

├── check_symbol.py

└── requirements.txt

- Each file handles a specific task
- Easy to maintain and extend
- Clear separation of logic

4. Integration Design

- Integration with MetaTrader 5 API for trading

- Real-time communication between Python and MT5
- Handles order execution and trade management

5. Error Handling Design

- Handles MT5 connection errors
- Manages trade execution failures
- Displays error messages in system
- Ensures stable and reliable performance

6. UI/UX Design

- Developed using Streamlit
- Simple and user-friendly interface
- Real-time data visualization
- Clean dashboard layout
- Easy trade execution controls

5.3 Interface Design

The system provides a simple and interactive interface using Streamlit.

- **Input:**
 - Selection of trading symbol (e.g., XAUUSD, EURUSD)
 - User actions (Buy / Sell / Close Trade)
 - Parameters like Stop Loss (SL) and Take Profit (TP)
- **Output:**
 - Real-time market data (price)
 - Technical indicators (EMA, RSI, MACD)
 - Buy/Sell signals
 - Trade status (open/closed)
 - Error or success messages

The interface is designed to be user-friendly and easy to operate.

5.3.1 State Transition

The system follows basic state transitions:

Idle → Data Fetching → Signal Generation → Trade Execution → Monitoring → Close Trade → Idle

- The system starts in idle state
- Moves to analysis and signal generation
- Executes trade if conditions are met
- Returns to idle after trade completion

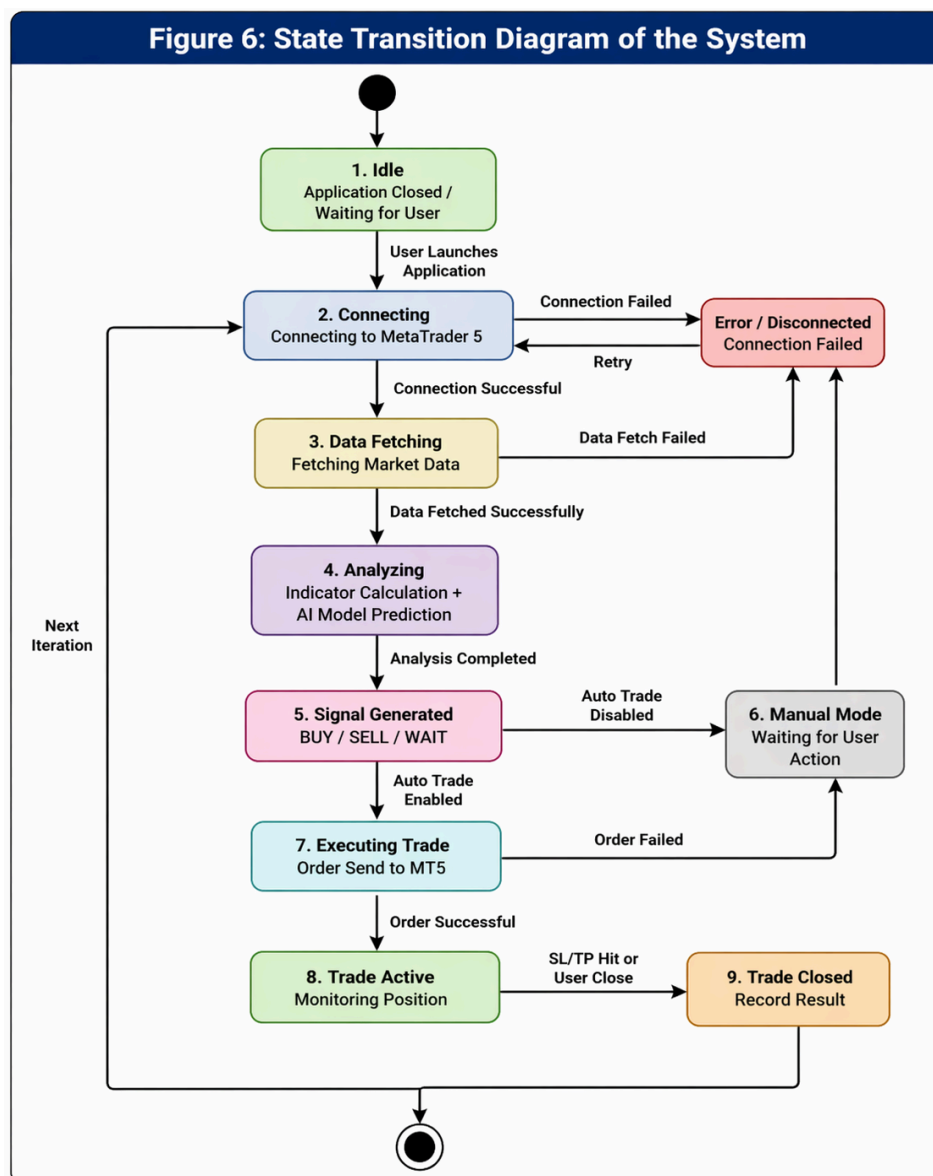


Figure 6 State Transition

5.3.2 Forms and Reports

Samples of Forms, Reports and Interface

- Dashboard screen showing market data and signals
- Trade execution panel (Buy/Sell buttons)
- Display of open trades and trade history
- Indicator charts (EMA, RSI, MACD)
- Status messages (success/error)

These components help users interact with the system easily.

5.3.3 Security Considerations

- Uses secure connection with MetaTrader 5
- No storage of sensitive user credentials in system
- Basic validation of trade inputs
- Error handling to prevent invalid trade execution
- System runs in demo trading environment for safety

6 IMPLEMENTATIONS

6.1 Environment

The system is implemented using a local development environment.

- **Operating System:** Windows
- **Programming Language:** Python
- **Framework:** Streamlit
- **Trading Platform:** MetaTrader 5 (Demo Account)
- **Tools Used:** VS Code / PyCharm
- **Libraries:** Pandas, TradingView TA, Requests

The system runs on a personal computer with internet connectivity for real-time data.

6.2 Technology Specifications

Component	Technology Used	Purpose
Programming Language	Python	Core development
UI Framework	Streamlit	Dashboard interface
Trading Platform	MetaTrader 5	Trade execution
Data Source	Yahoo Finance	Live market data
Signal Source	TradingView TA	Recommendations
Data Processing	Pandas	Analysis
Charts	Plotly	Visualization

ML Model	Random Forest	AI prediction
IDE	VS Code / PyCharm	Development

Table 6 Technology Specification

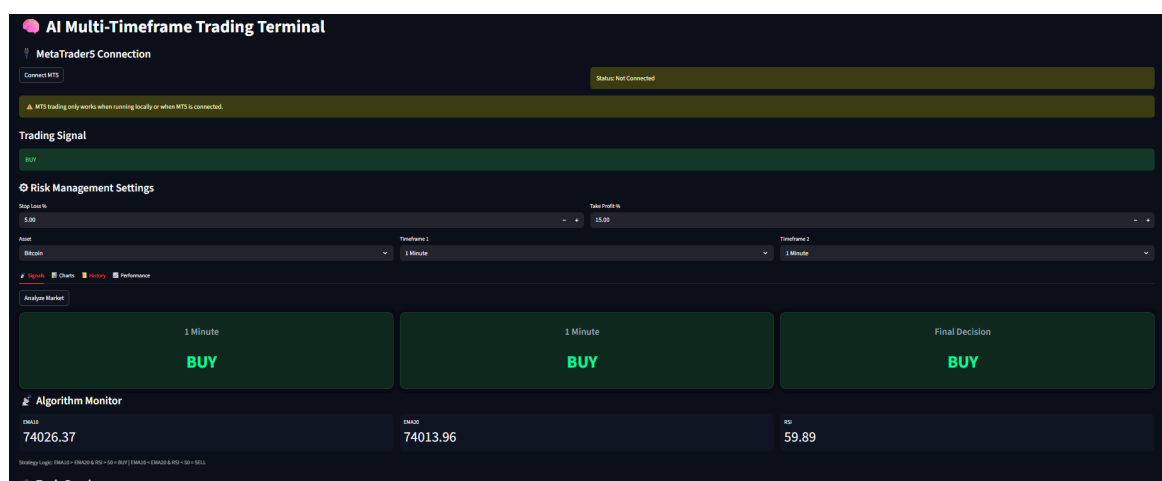
6.3 Results

The developed system produced successful results during testing and execution.

Outcomes:

- Successfully connected with MetaTrader 5 demo account
- Generated Buy / Sell / Wait signals accurately
- Real-time Bitcoin and Gold data displayed successfully
- Technical indicators calculated correctly
- Machine Learning model assisted signal generation
- Trades executed successfully in demo mode
- Dashboard displayed charts and performance metrics
- Risk management using Stop Loss / Take Profit worked correctly

The system demonstrated stable and efficient performance for learning and demo trading.



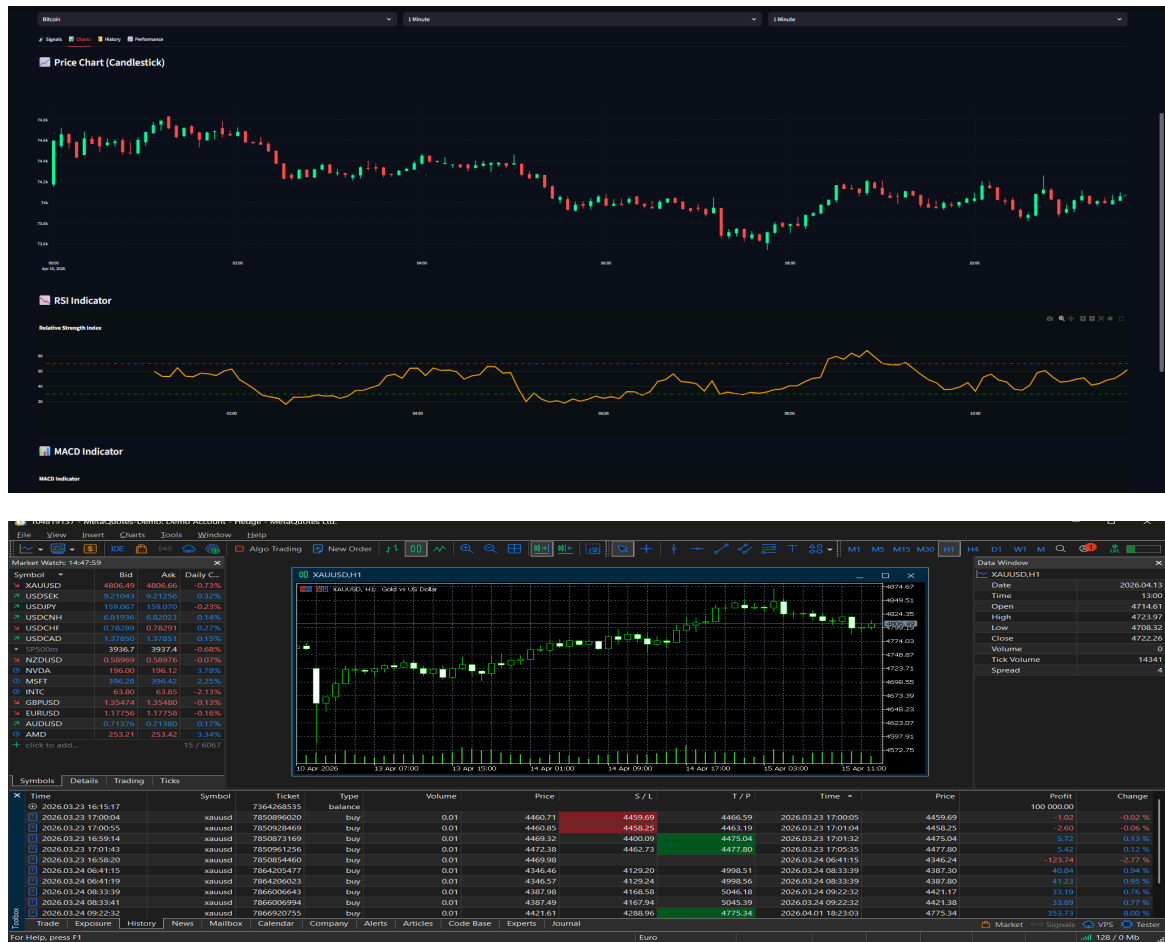


Figure 7 Dashboard

6.4 Result Analysis

1. Performance Analysis

- Fast real-time signal generation
- Smooth dashboard performance
- Quick MT5 trade execution
- Efficient data processing using Pandas

2. Usability Analysis

- Easy-to-use professional dashboard
- Simple navigation for trading controls

- Clear charts and signal display
- User-friendly interface

3. Functional Analysis

- AI signals generated successfully
- Technical indicators working correctly
- Trade open / close functionality successful
- Risk management system functional
- Auto trading logic operational

4. Comparison with Existing System

Criteria	Existing Manual Trading	Proposed AI Trading System
Speed	Slow	Fast
Accuracy	Human dependent	Logic + AI based
Monitoring	Limited	Real-time
Execution	Manual	Automated
Risk Control	Basic	SL / TP
Analysis	Manual	Multi-timeframe

Table 7 Comparison of System

7 TESTING

7.1 Plan

The testing strategy for the Modular Algo Trading Analysis System is designed to verify all functionalities, ensure system reliability, and provide accurate trading operations. The testing process follows a structured approach covering different levels of testing.

1. Testing Objectives

- To ensure all trading functionalities work correctly
- To identify and fix bugs and execution errors
- To verify real-time data processing and performance
- To ensure correct trade execution using MT5
- To improve overall system stability and usability

2. Types of Testing Performed

a) Unit Testing

- Testing individual modules
- Example: Indicator calculation (EMA, RSI, MACD), MT5 connection

b) Integration Testing

- Testing interaction between system modules and MT5
- Example: Signal generation → trade execution flow

c) System Testing

- Testing complete system functionality
- Example: End-to-end trading process (data → signal → trade)

d) Functional Testing

- Verifying all features work correctly
- Example: Buy/Sell execution, closing trades, signal display

e) User Interface (UI) Testing

- Checking dashboard layout and usability
- Ensuring proper display of data and controls

f) Performance Testing

- Checking response time for real-time data
- Ensuring smooth execution of trading operations

g) Security Testing

- Verifying safe connection with MT5
- Preventing invalid trade execution

3. Testing Approach

The following approach was used:

- Test each module individually (Unit Testing)
- Integrate modules and test complete workflow
- Identify and fix errors

Tool	Purpose
MetaTrader 5	Trade execution testing
Streamlit	UI testing
VS Code / PyCharm	Debugging and development
Manual Testing	Functional and system testing

Table 8 Technical approach

4. Testing Tools Used5. Test Data

- Real-time market data from MT5
- Sample trading symbols (XAUUSD, EURUSD)
- Demo trading account data

6. Acceptance Criteria

The system is considered successful if:

- All trading features work correctly
- No critical errors in trade execution
- Real-time data is processed accurately
- Dashboard is responsive and user-friendly
- System runs smoothly under normal conditions

7. Testing Challenges

- Handling real-time market data fluctuations
- Managing MT5 connection and execution errors
- Ensuring correct signal generation
- Debugging trade execution issues

7.2 Test Results

The system was tested using multiple scenarios covering data analysis, AI prediction, signal generation, and trade execution.

Overall Results:

- MT5 connection successful
- Signals generated correctly
- Buy/Sell trades executed successfully
- Stop Loss / Take Profit working correctly
- Real-time charts displayed properly
- Dashboard performed smoothly
- Minor bugs were fixed during testing
- No critical errors found in final testing

7.2.1 Test Cases

Test ID	Test Condition	Expected Output	Actual Output	Remark
TC01	Connect MT5	Connection Success	Connected	Pass
TC02	Load Dashboard	UI opens properly	Loaded	Pass
TC03	Fetch BTC Data	Live data displayed	Success	Pass
TC04	Generate Signal	Buy/Sell/Wait shown	Success	Pass
TC05	Execute Buy Trade	Trade opens	Success	Pass
TC06	Execute Sell Trade	Trade opens	Success	Pass
TC07	Close Trade	Trade closes	Success	Pass
TC08	SL/TP Check	Risk control active	Success	Pass
TC09	Chart Load	Charts visible	Success	Pass
TC10	Auto Trade Logic	Auto order works	Success	Pass

Table 9 Test Cases

8 CONCLUSION AND DISCUSSION

8.1 Overall Analysis

The internship project successfully demonstrates the implementation of a Modular Algo Trading Analysis System using Python, Streamlit, and MetaTrader 5. The system is capable of performing real-time market analysis, generating trading signals, and executing trades in a demo environment.

The project is technically feasible, cost-effective, and practical for learning algorithmic trading concepts. It provides a strong foundation for developing advanced trading systems and shows real-world applicability in financial technology.

8.2 Visit by Mentor



8.3 Evaluation Dates

Dates of Continuous Evaluation (CE-I and CE-II)

CE-I Evaluation Date: 25-02-2026

CE-II Evaluation Date: 16-04-2026

8.4 Problem and Solutions

Problem Encountered	Possible Solution
MT5 connection issues	Reinitialize connection and check login
Trade execution errors	Handle filling modes and validate inputs
Real-time data delay	Improve data fetching and refresh rate
Incorrect signal generation	Refine indicator logic
Debugging integration issues	Modular testing and step-by-step debugging

Table 10 Problems and Solutions

8.5 Summary

During the internship, I successfully developed a Modular Algo Trading Analysis System using Python, Streamlit, Machine Learning, and MetaTrader 5.

The project provides:

- Real-time market analysis
- AI-based signal generation
- Automated trading execution
- Risk management system
- Interactive dashboard

This internship enhanced my practical skills in Python development, financial systems, Machine Learning, API integration, and debugging real-world applications.

8.6 Future Scope

Limitations:

- Uses indicator-based logic (no advanced AI models)
- Limited risk management features
- Supports limited trading symbols

Future Enhancements:

- Integration of AI/ML models for better prediction
- Advanced risk management strategies
- Support for more trading instruments
- Mobile application development
- Real-time alerts and notifications

REFERENCES

1. MetaQuotes Ltd. (2024) *MetaTrader 5 User Guide and API Documentation*. Available at:
<https://www.metatrader5.com/en/terminal/help>
2. Streamlit Inc. (2024) *Streamlit Documentation*. Available at:
<https://docs.streamlit.io>
3. TradingView (2024) *TradingView Technical Analysis Library Documentation*. Available at:
<https://python-tradingview-ta.readthedocs.io/en/latest/>